BLOG REST API

Beginner-Friendly Backend Mini Project

Node.js + Express.js + MongoDB + Mongoose

JWT Authentication | HTTP-only Cookies | bcrypt | CRUD | Postman

---

Student guide

Start from an empty folder and build a complete, testable backend one step at a time. Each section explains what to create, which code to write, and how to test it.

> Learning goal: understand the request flow from Postman to Express routes, controllers, Mongoose models and MongoDB. No frontend is required.

Version: beginner learning project | Use Node.js LTS and a recent Postman version

How to use this guide

Follow the steps in order. Do not paste every file at once. Create one file, save it, run or test the relevant feature, and then continue.

- Step 1-4: install tools, create the project and configure MongoDB.
- Step 5-8: create models and build signup/login/logout.
- Step 9-12: protect routes, implement Blog CRUD and start the server.
- Step 13-15: test every endpoint in Postman and troubleshoot errors.

What you will build

Feature	What it does
Health API	Checks that the server is responding.
Signup	Creates a user and stores a bcrypt password hash.
Login	Checks credentials, signs a JWT and sets an HTTP-only cookie.
Logout	Clears the authentication cookie.
Blog CRUD	Create, read, update and delete blog posts.
Authorization	Only the author can update or delete their own blog.
Postman tests	Send requests and check status codes and JSON responses.

Basic words to know

Term	Simple meaning
Backend	Code that receives requests and returns responses.
API	A defined way for programs to communicate.
Route	The HTTP method and URL that select an action.
Controller	Function that performs the action for a request.
Model	Defines the shape of data stored in MongoDB.
Middleware	Function that runs between receiving a request and the controller.
CRUD	Create, Read, Update and Delete.
JWT	A signed token used here to identify a logged-in user.
Cookie	A small value sent with HTTP requests by a client.
HTTP-only	A cookie setting that prevents browser JavaScript from reading the cookie.

Step 1 - Install the required tools

1. Node.js

Install the current Node.js LTS release from https://nodejs.org/. Node.js runs JavaScript outside the browser.

```
node -v
npm -v
```

Both commands should print version numbers. If a command is not recognized, finish the installation and reopen the terminal.

2. MongoDB

Choose one option: run MongoDB Community Server locally, or create a database in MongoDB Atlas. For a local installation, the guide uses the connection string mongodb://127.0.0.1:27017/blogDB.

Atlas users must use their own connection string and configure database access/network access in Atlas. Never share the connection string publicly.

3. Code editor and Postman

Install VS Code (or another editor) and Postman. Postman will send HTTP requests directly to the backend; no frontend is needed.

Official downloads: Node.js - https://nodejs.org/ | MongoDB Community - https://www.mongodb.com/try/download/community | Atlas - https://www.mongodb.com/cloud/atlas | Postman - https://www.postman.com/downloads/

Step 2 - Create the Node.js project

Open a terminal in the location where you want to save the project. Run each command in order.

```
mkdir blog-rest-api
cd blog-rest-api
npm init -y
```

npm init -y creates package.json, which records project information and installed packages.

Install the packages

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cookie-parser
npm install --save-dev nodemon
```

Package	Why we need it
express	Creates the HTTP server and routes.
mongoose	Connects to MongoDB and defines schemas/models.
dotenv	Loads values from the .env file.
bcryptjs	Hashes passwords and checks submitted passwords.
jsonwebtoken	Signs and verifies JWTs.
cookie-parser	Makes request cookies available as req.cookies.
nodemon	Restarts the server after code changes during development.

Add the run commands

Open package.json. Keep its other generated fields and replace only the scripts section with:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Save the file. The command npm run dev will start the development server after server.js is created.

Create .gitignore

```
node_modules/
.env
```

> Do not upload node_modules or .env to GitHub. The .env file contains private configuration and secrets.

Step 3 - Create the folder structure

Create these folders and files in VS Code. Spelling and letter case matter.

```
blog-rest-api/
  config/
    db.js
  controllers/
    authController.js
    blogController.js
  middleware/
    authMiddleware.js
  models/
    User.js
    Blog.js
  routes/
    authRoutes.js
    blogRoutes.js
  .env
  .gitignore
  package.json
  server.js
```

What belongs where?

File/folder	Purpose
config/db.js	Connects to MongoDB.
models/User.js	Defines user data.
models/Blog.js	Defines blog data and its author.
controllers/authController.js	Signup, login and logout logic.
controllers/blogController.js	Create, read, update and delete logic.
middleware/authMiddleware.js	Checks the JWT cookie.
routes/authRoutes.js	Maps authentication URLs to functions.
routes/blogRoutes.js	Maps blog URLs to functions.
server.js	Loads middleware, registers routes and starts the server.

Step 4 - Configure environment variables

Create a file named .env in the project root (same level as server.js).

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/blogDB
JWT_SECRET=replace_with_a_long_random_secret
```

For MongoDB Atlas, replace the MONGO_URI value with the connection string provided by Atlas. Do not include quotation marks unless they are part of the actual value.

> JWT_SECRET is an example placeholder. For a real deployment, generate a long, random secret and keep it private. Never commit .env to source control.

Step 5 - Connect to MongoDB

Create config/db.js:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Explanation: mongoose.connect opens the database connection. async/await lets us wait for it. try/catch handles connection failures. module.exports lets server.js use this function.

Step 6 - Create the User model

Create models/User.js:

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: true,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

A User has a name, email and password hash. timestamps adds createdAt and updatedAt automatically. unique creates a uniqueness constraint/index; the controller must still handle duplicate-email errors.

Step 7 - Create the Blog model

Create models/Blog.js:

```
const mongoose = require("mongoose");

const blogSchema = new mongoose.Schema(
  {
    title: {
      type: String,
```

```
      required: true,
      trim: true,
    },
    content: {
      type: String,
      required: true,
    },
    author: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Blog", blogSchema);
```

The author field stores the MongoDB ID of the user who created the blog. This relationship lets the backend check ownership before an update or delete.

Step 8 - Build authentication: signup, login and logout

Create controllers/authController.js. The controller checks input, talks to the User model, hashes passwords, creates a JWT and sets or clears a cookie.

controllers/authController.js

```
const User = require("../models/User");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

// SIGNUP
const signup = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({
        message: "Name, email and password are required",
      });
    }

    if (password.length < 6) {
      return res.status(400).json({
        message: "Password must be at least 6 characters",
      });
    }

    const normalizedEmail = email.trim().toLowerCase();
    const existingUser = await User.findOne({ email: normalizedEmail });

    if (existingUser) {
      return res.status(409).json({
        message: "Email already registered",
      });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const user = await User.create({
      name: name.trim(),
      email: normalizedEmail,
      password: hashedPassword,
    });

    return res.status(201).json({
      message: "User registered successfully",
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
      },
    });
  } catch (error) {
    if (error.code === 11000) {
      return res.status(409).json({
        message: "Email already registered",
      });
    }

    return res.status(500).json({ message: "Signup failed" });
  }
};

// LOGIN
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        message: "Email and password are required",
      });
    }

    const user = await User.findOne({
      email: email.trim().toLowerCase(),
    });

    if (!user) {
      return res.status(401).json({
        message: "Invalid email or password",
      });
    }

    const isPasswordCorrect = await bcrypt.compare(
      password,
      user.password
    );
```

```
    if (!isPasswordCorrect) {
      return res.status(401).json({
        message: "Invalid email or password",
      });
    }

    const token = jwt.sign(
      { userId: user._id.toString() },
      process.env.JWT_SECRET,
      { expiresIn: "1d" }
    );

    res.cookie("token", token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "lax",
      maxAge: 24 * 60 * 60 * 1000,
    });

    return res.status(200).json({
      message: "Login successful",
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
      },
    });
  } catch (error) {
    return res.status(500).json({ message: "Login failed" });
  }
};

// LOGOUT
const logout = (req, res) => {
  res.clearCookie("token", {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
  });

  return res.status(200).json({
    message: "Logout successful",
  });
};

module.exports = { signup, login, logout };
```

Understand the key lines

- bcrypt.hash(password, 10) creates a password hash. Save the hash, never the plain password.
- bcrypt.compare checks a submitted password against the stored hash.
- jwt.sign creates a signed token with the user ID and a one-day expiration.
- httpOnly prevents browser JavaScript from reading the cookie.
- sameSite: lax is a useful basic setting, but production cookie authentication also needs suitable CSRF and CORS configuration.

Create routes/authRoutes.js

```
const express = require("express");
const {
  signup,
  login,
  logout,
} = require("../controllers/authController");

const router = express.Router();

router.post("/signup", signup);
router.post("/login", login);
router.post("/logout", logout);

module.exports = router;
```

These route definitions will become /api/auth/signup, /api/auth/login and /api/auth/logout when mounted in server.js.

Step 9 - Protect routes with authentication middleware

Create middleware/authMiddleware.js. This function runs before a protected controller.

```
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
  try {
    const token = req.cookies.token;

    if (!token) {
      return res.status(401).json({
        message: "Please login first",
      });
    }

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    req.userId = decoded.userId;
    next();
  } catch (error) {
    return res.status(401).json({
      message: "Invalid or expired token",
    });
  }
};

module.exports = authMiddleware;
```

How middleware works

- req.cookies.token reads the cookie named token.
- jwt.verify checks the token signature and expiration.
- req.userId stores the authenticated user's ID for the next function.
- next() continues to the controller only after verification succeeds.

A JWT is signed, not encrypted by default. Do not store passwords or other secrets in the token payload.

Step 10 - Create the Blog CRUD controller

Create controllers/blogController.js. This version includes basic input checks, invalid-ID handling, and an ownership check for update/delete.

```
const Blog = require("../models/Blog");

// CREATE BLOG
const createBlog = async (req, res) => {
  try {
    const { title, content } = req.body;

    if (
      typeof title !== "string" || !title.trim() ||
      typeof content !== "string" || !content.trim()
    ) {
      return res.status(400).json({
        message: "Title and content are required",
      });
    }

    const blog = await Blog.create({
      title: title.trim(),
      content: content.trim(),
      author: req.userId,
    });

    return res.status(201).json({
      message: "Blog created successfully",
      blog,
    });
  } catch (error) {
    return res.status(500).json({
      message: "Failed to create blog",
```

```
    });
  }
};

// READ ALL BLOGS
const getAllBlogs = async (req, res) => {
  try {
    const blogs = await Blog.find()
      .populate("author", "name email")
      .sort({ createdAt: -1 });

    return res.status(200).json({
      count: blogs.length,
      blogs,
    });
  } catch (error) {
    return res.status(500).json({
      message: "Failed to fetch blogs",
    });
  }
};

// READ ONE BLOG
const getBlogById = async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id)
      .populate("author", "name email");

    if (!blog) {
      return res.status(404).json({ message: "Blog not found" });
    }

    return res.status(200).json({ blog });
  } catch (error) {
    if (error.name === "CastError") {
      return res.status(400).json({ message: "Invalid blog ID" });
    }
    return res.status(500).json({ message: "Failed to fetch blog" });
  }
};

// UPDATE BLOG
const updateBlog = async (req, res) => {
  try {
    const { title, content } = req.body;

    if (title === undefined && content === undefined) {
      return res.status(400).json({
        message: "Provide a title or content to update",
      });
    }

    if (
      title !== undefined &&
      (typeof title !== "string" || !title.trim())
    ) {
      return res.status(400).json({
        message: "Title must be a non-empty string",
      });
    }

    if (
      content !== undefined &&
      (typeof content !== "string" || !content.trim())
    ) {
      return res.status(400).json({
        message: "Content must be a non-empty string",
      });
    }

    const blog = await Blog.findById(req.params.id);

    if (!blog) {
      return res.status(404).json({ message: "Blog not found" });
    }

    if (blog.author.toString() !== req.userId) {
      return res.status(403).json({
        message: "You can update only your own blogs",
      });
    }

    if (title !== undefined) blog.title = title.trim();
    if (content !== undefined) blog.content = content.trim();
```

```
    await blog.save();

    return res.status(200).json({
      message: "Blog updated successfully",
      blog,
    });
  } catch (error) {
    if (error.name === "CastError") {
      return res.status(400).json({ message: "Invalid blog ID" });
    }
    return res.status(500).json({ message: "Failed to update blog" });
  }
};

// DELETE BLOG
const deleteBlog = async (req, res) => {
  try {
    const blog = await Blog.findById(req.params.id);

    if (!blog) {
      return res.status(404).json({ message: "Blog not found" });
    }

    if (blog.author.toString() !== req.userId) {
      return res.status(403).json({
        message: "You can delete only your own blogs",
      });
    }

    await blog.deleteOne();

    return res.status(200).json({
      message: "Blog deleted successfully",
    });
  } catch (error) {
    if (error.name === "CastError") {
      return res.status(400).json({ message: "Invalid blog ID" });
    }
    return res.status(500).json({ message: "Failed to delete blog" });
  }
};

module.exports = {
  createBlog,
  getAllBlogs,
  getBlogById,
  updateBlog,
  deleteBlog,
};
```

Step 11 - Create Blog routes

Create routes/blogRoutes.js. Public users can read blogs. A logged-in user is required to create, update or delete.

```
const express = require("express");
const router = express.Router();

const authMiddleware = require("../middleware/authMiddleware");

const {
  createBlog,
  getAllBlogs,
  getBlogById,
  updateBlog,
  deleteBlog,
} = require("../controllers/blogController");

// Public routes
router.get("/", getAllBlogs);
router.get("/:id", getBlogById);

// Protected routes: middleware runs before controller
router.post("/", authMiddleware, createBlog);
router.put("/:id", authMiddleware, updateBlog);
router.delete("/:id", authMiddleware, deleteBlog);

module.exports = router;
```

For example, router.post('/', authMiddleware, createBlog) checks authentication first. If the token is valid, the createBlog controller runs.

Step 12 - Connect everything in server.js

Create server.js in the project root:

```
const express = require("express");
const dotenv = require("dotenv");
const cookieParser = require("cookie-parser");

const connectDB = require("./config/db");

dotenv.config();

if (!process.env.JWT_SECRET) {
  throw new Error("JWT_SECRET is missing from .env");
}

const app = express();

// Middleware
app.use(express.json());
app.use(cookieParser());

// Health API
app.get("/api/health", (req, res) => {
  res.status(200).json({
    message: "Blog API is running",
  });
});

// Register routes
app.use("/api/auth", require("./routes/authRoutes"));
app.use("/api/blogs", require("./routes/blogRoutes"));

// Connect to MongoDB before listening for requests
const startServer = async () => {
  try {
    await connectDB();

    const PORT = process.env.PORT || 5000;

    app.listen(PORT, () => {
      console.log(`Server running at http://localhost:${PORT}`);
    });
  } catch (error) {
    console.error("Failed to start server:", error.message);
    process.exit(1);
  }
};
```

```
startServer();
```

Why does order matter?

- dotenv.config() must run before reading environment variables.
- express.json() must be registered before routes so JSON bodies can be read.
- cookieParser() must run before routes that read req.cookies.
- The server connects to MongoDB before accepting requests.

Step 13 - Run the server

```
npm run dev
```

Expected terminal output:

```
MongoDB connected successfully
Server running at http://localhost:5000
```

If the database connection fails, check that MongoDB is running or that your Atlas URI is correct. Do not continue to protected API testing until the server starts successfully.

Step 14 - Test APIs in Postman

Create a Postman request for each endpoint below. For JSON request bodies, select Body > raw > JSON. Use http://localhost:5000 as the base URL.

Test 1 - Health API

Method	URL	Body
GET	http://localhost:5000/api/health	None

Expected status: 200 OK

```
{
  "message": "Blog API is running"
}
```

Test 2 - Signup

Method	URL	Body
POST	http://localhost:5000/api/auth/signup	{"name":"Rohan","email":"rohan@example.com","password":"rohan123"}

Expected status: 201 Created. The response returns the user ID, name and email, but not the password.

```
{
  "name": "Rohan",
  "email": "rohan@example.com",
  "password": "rohan123"
}
```

Try the same email again. Expected result: 409 Conflict.

Test 3 - Login

Method	URL	Body
POST	http://localhost:5000/api/auth/login	{"email":"rohan@example.com","password":"rohan123"}

```
{
  "email": "rohan@example.com",
  "password": "rohan123"
}
```

Expected status: 200 OK and a Set-Cookie response header for token. Check Postman's cookie manager to confirm the cookie is saved.

Cookie testing notes

- Use localhost consistently. Do not switch between localhost and 127.0.0.1 during the test sequence.
- Postman usually manages cookies automatically. If a protected request returns 401, inspect the cookie manager and request headers.
- Do not manually put the JWT in JSON. This project authenticates protected requests with the token cookie.

Test 4 - Create a blog

Method	URL	Body
POST	http://localhost:5000/api/blogs	{"title":"My First Blog","content":"I am learning Express."}

```
{
  "title": "My First Blog",
  "content": "I am learning Express."
}
```

Expected status: 201 Created. Run this only after login, so Postman sends the token cookie. The author is taken from the verified token, not from the request body.

Test 5 - Get all blogs

Method	URL	Body
GET	http://localhost:5000/api/blogs	None

Expected status: 200 OK. The response contains count and a blogs array. Each blog's author is populated with name and email.

Test 6 - Get one blog

Method	URL	Body
GET	http://localhost:5000/api/blogs/BLOG_ID	None

Replace BLOG_ID with the actual _id returned by the create-blog response. Expected status: 200 OK.

Step 14 continued - Update, delete and logout

Test 7 - Update a blog

Method	URL	Body
PUT	http://localhost:5000/api/blogs/BLOG_ID	{"title":"Updated Blog","content":"I finished the project."}

```
{
  "title": "Updated Blog",
  "content": "I finished the project."
}
```

Expected status: 200 OK. Only the author can update the blog. Replace BLOG_ID with the actual blog ID.

Test 8 - Delete a blog

Method	URL	Body
DELETE	http://localhost:5000/api/blogs/BLOG_ID	None

Expected status: 200 OK.

```
{
  "message": "Blog deleted successfully"
}
```

Test 9 - Logout

Method	URL	Body
POST	http://localhost:5000/api/auth/logout	None

```
{
  "message": "Logout successful"
}
```

The server clears the token cookie. Try creating a blog again; it should return 401 Unauthorized once the cookie is gone.

> Logout clears the client cookie, but a copied JWT remains valid until expiration unless the application adds token revocation or server-side sessions. That advanced feature is outside this beginner project.

Complete API checklist

Method	Endpoint	Access	Purpose
GET	/api/health	Public	Check server
POST	/api/auth/signup	Public	Register user
POST	/api/auth/login	Public	Login and set cookie
POST	/api/auth/logout	Public	Clear cookie
POST	/api/blogs	Login required	Create blog
GET	/api/blogs	Public	Get all blogs
GET	/api/blogs/:id	Public	Get one blog
PUT	/api/blogs/:id	Owner only	Update blog
DELETE	/api/blogs/:id	Owner only	Delete blog

Base URL: http://localhost:5000. For a full URL, add the base URL to the endpoint, for example http://localhost:5000/api/blogs.

Step 15 - HTTP status codes to recognize

Status	Meaning in this project
200 OK	Request succeeded.
201 Created	A user or blog was created.
400 Bad Request	Required input is missing, invalid, or the ID is malformed.
401 Unauthorized	Login is required or the token is invalid/expired.
403 Forbidden	Logged-in user does not own this blog.
404 Not Found	The requested blog does not exist.
409 Conflict	The email is already registered.
500 Internal Server Error	Unexpected server-side failure.

Troubleshooting guide

Problem	What to check
Cannot find module	Check spelling, file names, folder names and require paths.
MongoDB connection failed	Make sure the local database is running or verify Atlas URI and access rules.
req.body is undefined/empty	Select Body > raw > JSON in Postman and confirm express.json() is before routes.
req.cookies is undefined	Check cookie-parser installation and app.use(cookieParser()) before routes.
401 on create/update/delete	Log in first; inspect Postman's cookie manager and confirm token is sent.
409 on signup	That email may already exist. Use a different test email.
Invalid blog ID	Copy the actual _id from a blog response.
403 on update/delete	The currently logged-in user is not the blog author.
JWT_SECRET missing	Check .env is in the project root and restart npm run dev.
Port already in use	Stop the other process or change PORT in .env.

Student practice tasks

- Register a second user with a different email.
- Log in as User A and create a blog.
- Log in as User B and try to update User A's blog. Expected: 403 Forbidden.
- Try creating a blog without logging in. Expected: 401 Unauthorized.
- Try signing up with a missing password. Expected: 400 Bad Request.
- Update only the title, leaving content unchanged.
- Delete a blog and then try to fetch it. Expected: 404 Not Found.
- Explain the difference between authentication (who are you?) and authorization (what are you allowed to do?).

Final project checklist

- [ ] Node.js and npm work in the terminal
- [ ] Required packages are installed
- [ ] MongoDB connects successfully
- [ ] Health endpoint returns 200
- [ ] User and Blog models are created
- [ ] Signup stores a bcrypt hash, not a plain password
- [ ] Login verifies the password and sets a JWT cookie
- [ ] Middleware rejects missing/invalid tokens
- [ ] Create and read blog APIs work
- [ ] Update and delete work for the blog author
- [ ] Other users cannot update or delete someone else's blog
- [ ] Logout clears the cookie
- [ ] Every endpoint has been tested in Postman

Final notes

This is a beginner teaching project, not a production-ready authentication system. Before deploying a real application, add stronger schema/input validation, request rate limiting, centralized error handling, appropriate CORS and CSRF protection, secure deployment secrets, logging, and automated tests.

Teaching tip: build one file at a time and test each endpoint before moving to the next feature. Ask students to predict the response before clicking Send in Postman.

End of guide